

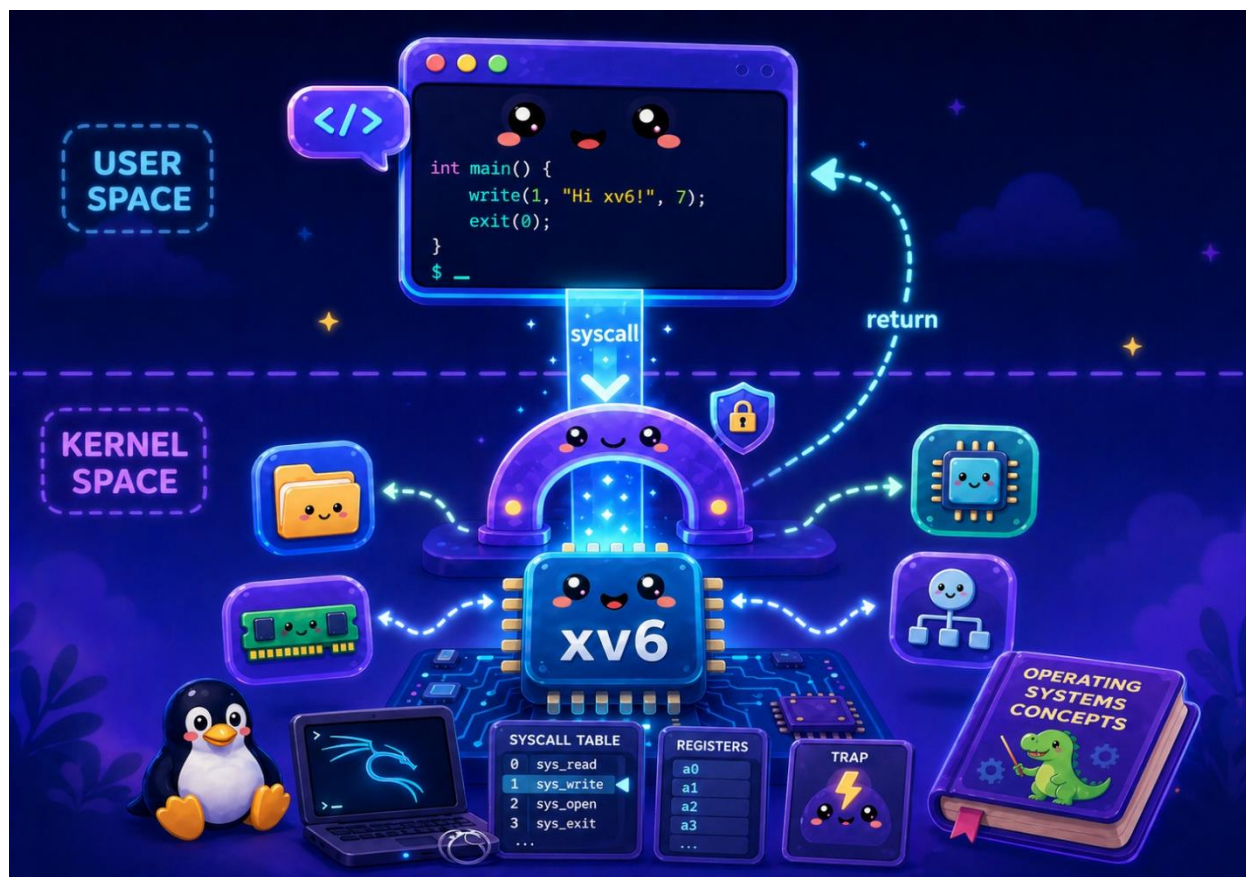


به نام خدا

آزمایشگاه سیستم عامل - بهار ۱۴۰۵

پروژه دوم: فراخوانی سیستمی

طراحان: یاسمن عموجعفری، فائزه بیات



اهداف پروژه

- آشنایی با سازوکار و چگونگی صدا زده شدن فراخوانی‌های سیستمی¹ در هسته xv6
- آشنایی با پیاده‌سازی تعدادی فراخوانی سیستمی در هسته xv6
- ذخیره سازی اطلاعات فراخوانی‌های سیستمی

¹System Call

- آشنایی با نحوه ذخیره‌سازی پردازنده‌ها و ساختار داده‌های مربوط به آن

مقدمه

هر برنامه در حال اجرا یک پردازنده² نام دارد. به این ترتیب یک سیستم کامپیوتری ممکن است در آن واحد، چندین پردازنده در انتظار سرویس داشته باشد. هنگامی که یک پردازنده در سیستم در حال اجرا است، پردازنده روال معمول پردازش را طی میکند: خواندن یک دستور، افزودن مقدار شمارنده برنامه³ به میزان یک واحد، اجرای دستور و نهایتاً تکرار حلقه. در یک سیستم رویدادهایی وجود دارند که باعث می‌شوند به جای اجرای دستور بعدی، کنترل از سطح کاربر به سطح هسته منتقل شود. به عبارت دیگر، هسته کنترل را در دست گرفته و به برنامه‌های سطح کاربر سرویس می‌دهد:⁴

- (1) ممکن است داده‌ای از دیسک دریافت شده باشد و به دلایلی لازم باشد بلافاصله آن داده از ثبات⁵ مربوطه در دیسک به حافظه منتقل گردد. انتقال جریان کنترل در این حالت، ناشی از وقفه⁶ خواهد بود. وقفه به طور غیر همگام با کد در حال اجرا رخ می‌دهد.
- (2) ممکن است یک استثنا⁷ مانند تقسیم بر صفر رخ دهد. در اینجا برنامه دارای یک دستور تقسیم بوده که عملوند مخرج آن مقدار صفر داشته و اجرای آن کنترل را به هسته می‌دهد.
- (3) ممکن است برنامه نیاز به عملیات ممتاز داشته باشد. عملیاتی مانند دسترسی به اجزای سخت افزاری یا حالت ممتاز سیستم (مانند محتوای ثبات‌های کنترلی) که تنها هسته اجازه دسترسی به آنها را دارد. در این شرایط برنامه اقدام به فراخوانی **فراخوانی سیستمی** می‌کند. طراحی سیستم‌عامل باید به گونه‌ای باشد که مواردی از قبیل ذخیره‌سازی اطلاعات پردازنده و بازیابی اطلاعات رویداد به وقوع پیوسته مثل آرگومان‌ها را به صورت ایزوله‌شده از سطح کاربر انجام دهد. در این پروژه، تمرکز بر روی فراخوانی سیستمی است.

²Process

³Program Counter

⁴ در 6xv به تمامی این موارد trap گفته میشود. در حالی که در حقیقت در 86x نام‌های متفاوتی برای این گذارها به کار می‌رود.

⁵Register

⁶Interrupt

⁷Exception

در اکثریت قریب به اتفاق موارد، فراخوانی‌های سیستمی به طور غیرمستقیم و توسط توابع کتابخانه‌ای پوشاننده⁸ مانند توابع موجود در کتابخانه استاندارد C در لینوکس یعنی glibc صورت می‌پذیرد.⁹ به این ترتیب قابلیت‌حمل¹⁰ برنامه‌های سطح کاربر افزایش می‌یابد. زیرا به عنوان مثال چنانچه در ادامه مشاهده خواهد شد، فراخوانی‌های سیستمی با شماره‌هایی مشخص می‌شوند که در معماری‌های مختلف، متفاوت است. توابع پوشاننده کتابخانه‌ای، این وابستگی‌ها را مدیریت می‌کنند. توابع پوشاننده 6xv در فایل usys.S توسط ماکروی SYSCALL تعریف شده‌اند.

پرسش 1: کتابخانه‌های سطح کاربر، موجود در دایرکتوری ULIB (مانند فایل‌های usys.S و ulib.c)، توابع پوشاننده‌ای را ارائه می‌دهند که این فراخوانی‌های سیستمی را به صورت انتزاعی مدیریت می‌کنند. توضیح دهید که این کتابخانه‌ها چگونه با استفاده از ماکروها و توابع پوشاننده، جزئیات فراخوانی‌های سیستمی (مانند شماره فراخوانی، آرگومان‌ها و بازگردانی مقادیر) را از برنامه‌نویس پنهان می‌کنند. همچنین، دلایل استفاده از این انتزاع در بهبود قابلیت حمل، افزایش امنیت و ساده‌سازی توسعه برنامه‌های کاربر را بیان نمایید.

تعداد فراخوانی‌های سیستمی، وابسته به سیستم عامل و حتی معماری پردازنده است. به عنوان مثال در لینوکس، فری‌بی‌اس‌دی¹¹ و ویندوز 7 به ترتیب حدود 300، 500 و 700 فراخوانی سیستمی وجود داشته که بسته به معماری پردازنده اندکی متفاوت خواهد بود [1]. در حالی که 6xv تنها 21 فراخوانی سیستمی دارد.

فراخوانی سیستمی سربارهایی دارد: (1) سربار مستقیم که ناشی از تغییر مد اجرایی و انتقال به مد ممتاز بوده و (2) سربار غیر مستقیم که ناشی از آلودگی ساختارهای پردازنده شامل انواع حافظه‌های نهان¹² و خط لوله¹³ می‌باشد. به عنوان مثال، در یک فراخوانی سیستمی write() در لینوکس تا $\frac{2}{3}$

⁸ Wrapper

⁹ در glibc، توابع پوشاننده غالباً نام و پارامترهایی مشابه فراخوانی‌های سیستمی دارند.

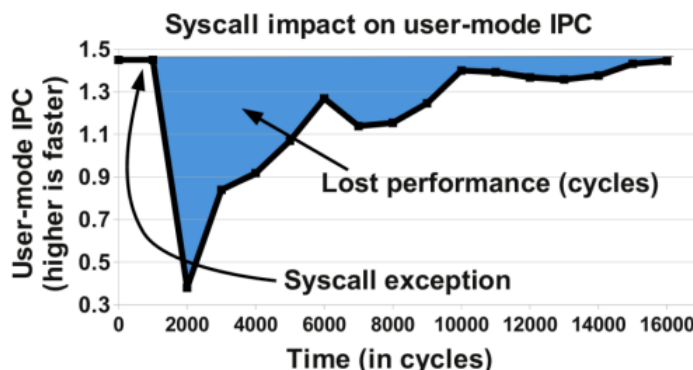
¹⁰ Portability

¹¹ FreeBSD

¹² Caches

¹³ Pipeline

حافظه نهان سطح یک داده خالی خواهد شد [2]. به این ترتیب ممکن است کارایی به نصف کاهش یابد. غالباً عامل اصلی، سربرار غیر مستقیم است. تعداد دستورالعمل اجرا شده به ازای هر سیکل¹⁴ هنگام اجرای یک فراخوانی سیستمی در بار کاری 2006 SPEC CPU روی پردازنده 7Intel Core i نمودار زیر نشان داده شده است [2].



مشاهده می‌شود که در لحظه‌ای IPC به کمتر از 0.4 رسیده است. روش‌های مختلفی برای فراخوانی سیستمی در پردازنده‌های 86x استفاده می‌گردد. روش قدیمی که در 6xv به کار می‌رود استفاده از دستور اسمبلی `int` است. مشکل اساسی این روش، سربرار مستقیم آن است. در پردازنده‌های مدرن‌تر 86x دستورهای اسمبلی جدیدی با سربرار انتقال کمتر مانند `sysexit/sysenter` ارائه شده است. در لینوکس، `glibc` در صورت پشتیبانی پردازنده، از این دستورها استفاده می‌کند. برخی فراخوانی‌های سیستمی (مانند `gettimeofday()` در لینوکس) فرکانس دسترسی بالا و پردازش کمی در هسته دارند. لذا سربرار مستقیم آن‌ها بر برنامه زیاد خواهد بود. در این موارد می‌توان از روش‌های دیگری مانند اشیای مجزای پویای مشترک¹⁵ در لینوکس بهره برد. به این ترتیب که هسته، پیاده‌سازی فراخوانی‌های سیستمی را در فضای آدرس سطح کاربر نگاشت داده و تغییر مد به مد هسته صورت نمی‌پذیرد. این دسترسی نیز به طور غیرمستقیم و توسط کتابخانه `glibc` صورت می‌پذیرد. در ادامه سازوکار اجرای فراخوانی سیستمی در 6xv مرور خواهد شد.

¹⁴ Instruction per Cycle

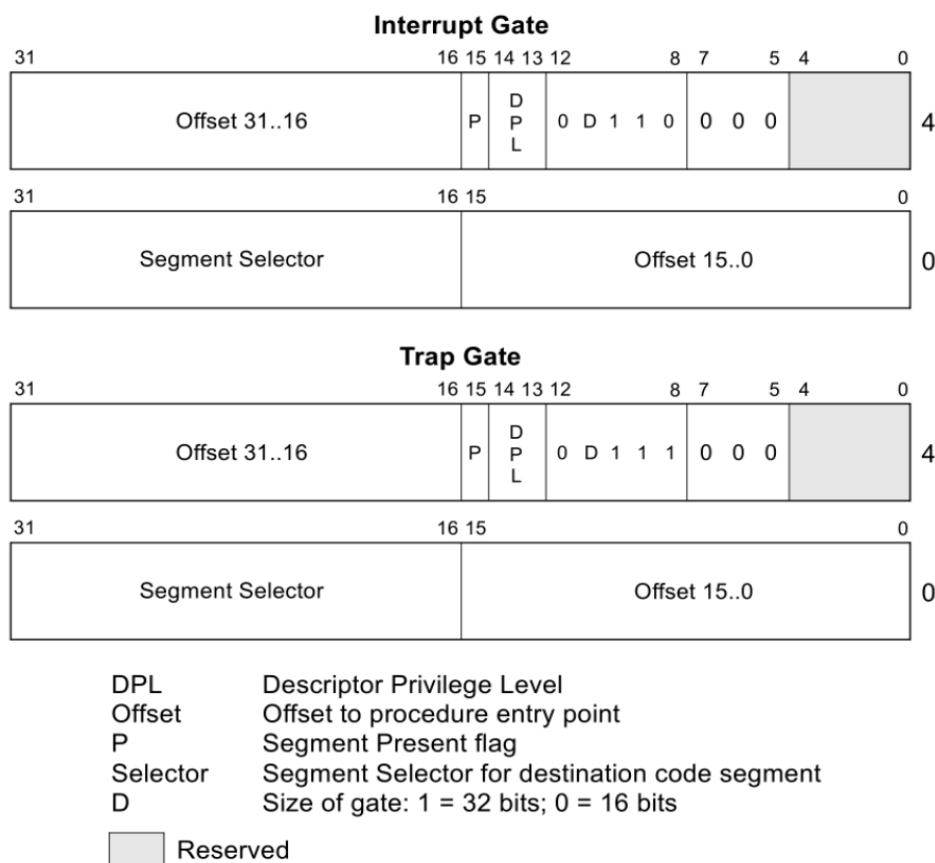
¹⁵ Virtual Dynamic Shared Objects (vDSO)

پرسش 2: فراخوانی‌های سیستمی تنها روش برای تعامل برنامه‌های کاربر با کرنل نیستند. چه روش‌های دیگری در لینوکس وجود دارند که برنامه‌های سطح کاربر می‌توانند از طریق آنها به کرنل دسترسی داشته باشند؟ هر یک از این روش‌ها را به اختصار توضیح دهید.

سازوکار اجرای فراخوانی سیستمی در xv6

بخش سخت‌افزاری و اسمبلی

جهت فراخوانی سیستمی در 6xv از روش قدیمی پردازنده‌های 86x استفاده می‌شود. در این روش، دسترسی به کد دارای سطح دسترسی ممتاز (در اینجا کد هسته) مبتنی بر مجموعه توصیفگرهایی موسوم به Gate Descriptor است. چهار نوع Gate Descriptor وجود دارد که 6xv تنها از Trap Gate و Interrupt Gate استفاده می‌کند. ساختار این Gate ها در شکل زیر نشان داده شده است [4].



این ساختارها در 6xv در قالب یک ساختار هشت بایتی موسوم به struct gatedesc تعریف شده‌اند (خط 855). به ازای هر انتقال به هسته (فراخوانی سیستمی و هر یک از انواع وقفه‌های سخت‌افزاری و استثناها) یک Gate در حافظه تعریف شده و یک شماره تله¹⁶ نسبت داده می‌شود. این Gate ها توسط تابع tvinit() در حین بوت (خط 1229) مقداردهی می‌گردند. Interrupt Gate اجازه وقوع وقفه در پردازنده حین کنترل وقفه را نمی‌دهد. در حالی که Trap gate اینگونه نیست. لذا برای فراخوانی سیستمی از Trap gate استفاده می‌شود تا وقفه که اولویت بیشتری دارد، همواره قابل سرویس‌دهی باشد (خط 3373). عملکرد Gate ها را می‌توان با بررسی پارامترهای ماکروی مقدار دهنده به Gate مربوط به فراخوانی سیستمی بررسی نمود:

پارامتر 1: T_SYSCALL[id] محتوای Gate مربوط به فراخوانی سیستمی را نگه می‌دارد. آرایه idt (خط ۳۳۶۱) بر اساس شماره تله‌ها اندیس‌گذاری شده است. پارامترهای بعدی، هر یک بخشی از T_SYSCALL[id] را پر می‌کنند.

پارامتر ۲: تعیین نوع Gate که در اینجا Gate Trap بوده و لذا مقدار یک دارد.

پارامتر ۳: نوع قطعه کدی که بلافاصله پس از اتمام عملیات تغییر مد پردازنده اجرا می‌گردد. کد کنترل‌کننده فراخوانی سیستمی در مد هسته اجرا خواهد شد. لذا مقدار KCODE_SEG >> 3 به ماکرو ارسال شده است.

پارامتر ۴: محل دقیق کد در هسته که vector[T_SYSCALL] است. این نیز بر اساس شماره تله‌ها شاخص‌گذاری شده است.

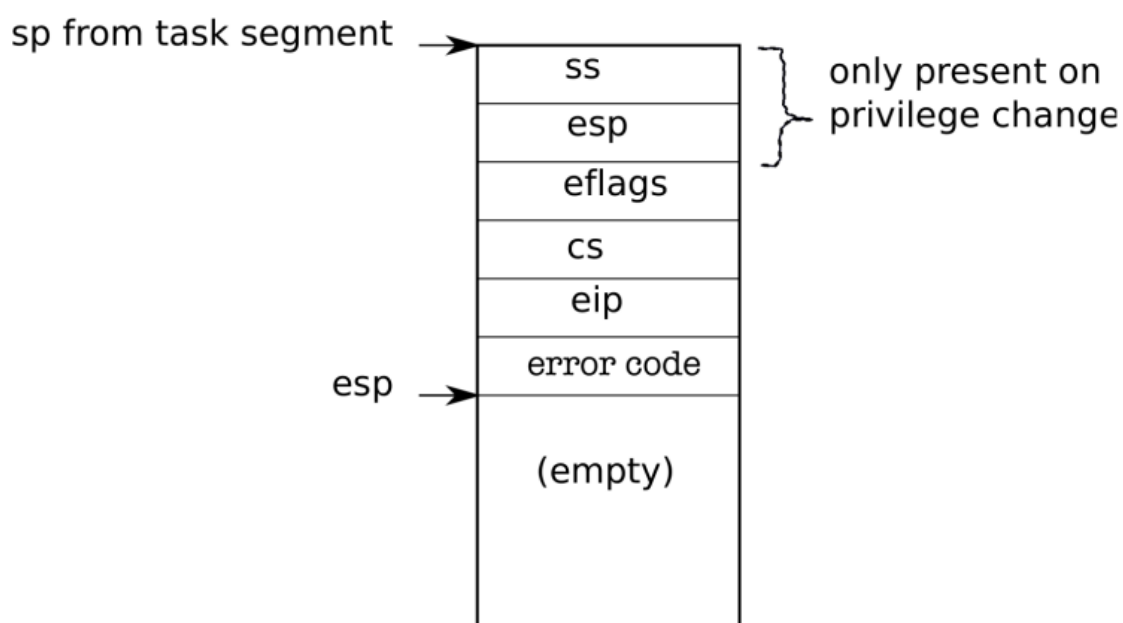
پارامتر ۵: سطح دسترسی مجاز برای اجرای این تله، DPL_USER است. زیرا فراخوانی سیستمی توسط (قطعه) کد سطح کاربر فراخوانی می‌گردد.

پرسش 3: با توجه به توضیحات ارائه‌شده درباره‌ی Gate Descriptor ها در 6xv و نحوه‌ی تنظیم سطح دسترسی برای فراخوانی‌های سیستمی، چرا تنها فراخوانی سیستمی (که با Trap Gate پیاده‌سازی شده) با سطح دسترسی DPL_USER فعال می‌شود و سایر تله‌ها (مانند وقفه‌های سخت‌افزاری و استثناها) نمی‌توانند از این سطح دسترسی بهره ببرند؟

¹⁶Trap Number

به این ترتیب برای تمامی تله‌ها idt مربوطه ایجاد می‌گردد. به عبارت دیگر، پس از اجرای tvinit() آرایه idt به طور کامل مقداردهی شده است. حال باید هر هسته پردازنده بتواند از اطلاعات idt استفاده کند تا بداند هنگام اجرای هر تله چه کد مدیریتی باید اجرا شود. بدین منظور، تابع idtinit() در انتهای راه‌اندازی اولیه هر هسته پردازنده اجرا شده و اشاره‌گر به جدول idt را در ثبات مربوطه در هر هسته بارگذاری می‌نماید. از این به بعد امکان سرویس‌دهی به تله‌ها فراهم است؛ یعنی پردازنده می‌داند برای هر تله چه کدی را فراخوانی کند.

یکی از راه‌های فعال‌سازی هر تله استفاده از دستور `<trap no init>` می‌باشد. لذا با توجه به این که شماره تله فراخوانی سیستمی ۶۴ است (خط ۳۲۲۶)، کافی است برنامه، جهت فراخوانی فراخوانی سیستمی، دستور `int 64` را فراخوانی کند. `int` یک دستورالعمل پیچیده در پردازنده `x86` (یک پردازنده CISC) است. ابتدا باید وضعیت پردازنده در حال اجرا ذخیره شود تا بتوان پس از فراخوانی سیستمی وضعیت را در سطح کاربر بازیابی نمود. اگر تله ناشی از خطا باشد (مانند خطای نقص صفحه که در فصل مدیریت حافظه معرفی می‌گردد)، کد خطا نیز در انتها روی پشته قرار داده می‌شود. پس از اتمام عملیات سخت‌افزاری مربوط به تله، حالت پشته (سطح هسته) در دستور `int` (مستقل از نوع تله با فرض Push شدن کد خطا توسط پردازنده) در شکل زیر نشان داده شده است. دقت شود مقدار `esp` با Push کردن کاهش می‌یابد.

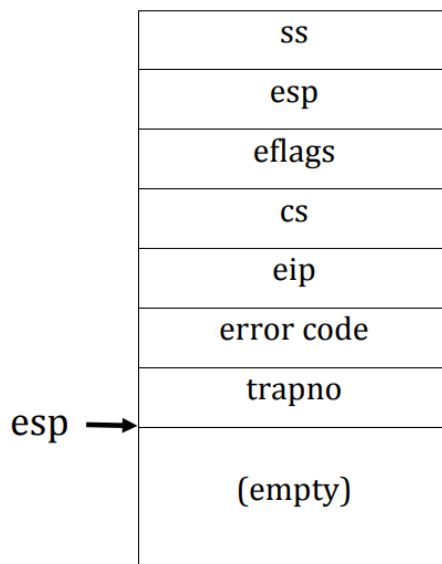


پرسش 4: در صورت تغییر سطح دسترسی، ss و esp روی پشته Push میشود. در غیراینصورت Push نمیشود. چرا؟

در آخرین گام `int`، بردار تله یا همان کد کنترل کننده مربوط به فراخوانی سیستمی اجرا می گردد که در شکل زیر نشان داده شده است.

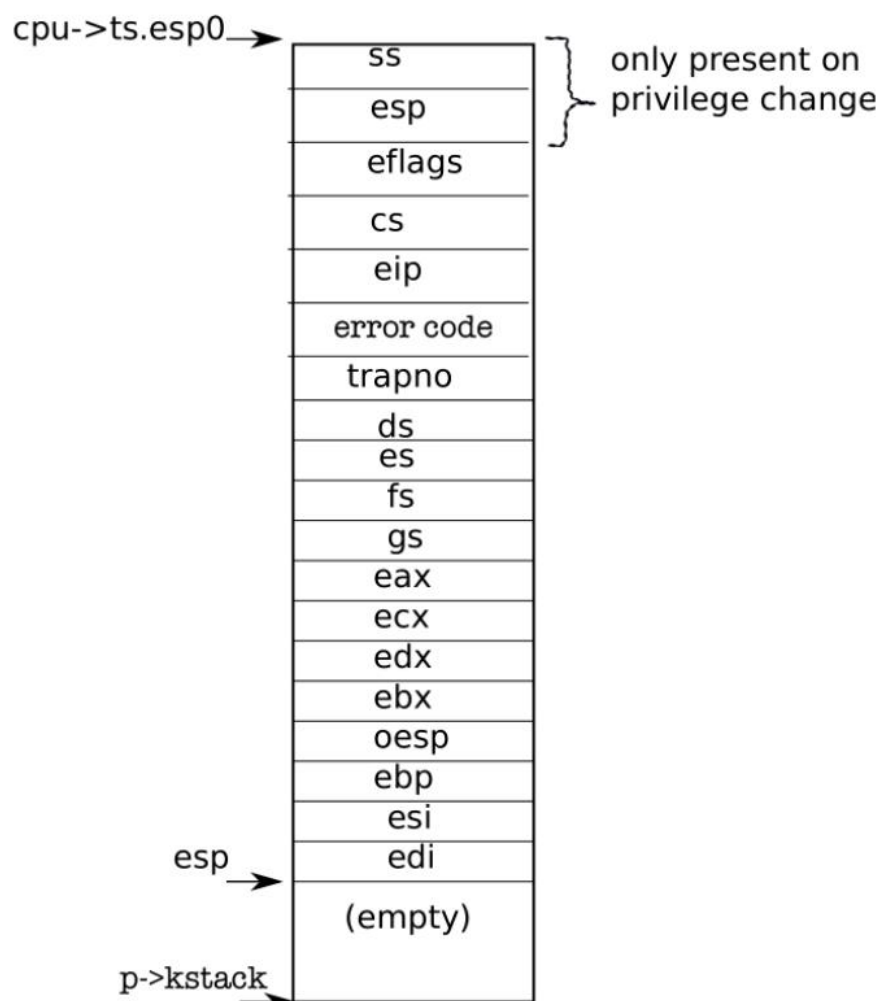
```
.globl vector64
vector64:
pushl $0
pushl $64
jmp alltraps
```

در اینجا ابتدا یک کد خطای بی اثر صفر و سپس شماره تله روی پشته قرار داده شده است. در انتها اجرا از کد اسمبلی `alltraps` ادامه می یابد. حالت پشته، پیش از اجرای کد `alltraps` در شکل زیر نشان داده شده است.



`alltraps` باقی ثبات ها را Push می کند. به این ترتیب، تمامی وضعیت برنامه سطح کاربر پیش از فراخوانی سیستمی ذخیره شده و قابل بازیابی است. شماره فراخوانی سیستمی و پارامترهای آن نیز در این وضعیت ذخیره شده و حضور دارند. این اطلاعات موجود در پشته همان قاب تله هستند که در

پروژه قبل مشابه آن برای برنامه `initcode.S` ساخته شده بود. حال اشاره گر به بالای پشته (`esp`) که در اینجا اشاره گر به قاب تله است، روی پشته قرار داده شده (خط ۳۳۲۸) و تابع `trap()` فراخوانی می شود. این معادل اسمبلی این است که اشاره گر به قاب تله به عنوان پارامتر به `trap()` ارسال شود. حالت پشته پیش از اجرای `trap()` در شکل زیر نشان داده شده است.



بخش سطح بالا و کنترل کننده زبان سی تله

تابع `trap` ابتدا نوع تله را با بررسی مقدار شماره تله چک میکند (خط ۳۴۰۳). با توجه به این که فراخوانی سیستمی رخ داده است تابع `syscall` اجرا میشود. پیشتر ذکر شد فراخوانی های سیستمی، متنوع بوده و هر یک دارای شمارهای منحصر به فرد است. این شماره ها در فایل `syscall.h` به فراخوانی های سیستمی نگاشت داده شده اند (خط ۳۵۰۰).

تابع (syscall) ابتدا وجود فراخوانی سیستمی فراخوانی شده را بررسی نموده و در صورت وجود پیاده سازی، آن را از جدول فراخوانی های سیستمی اجرا می کند. جدول فراخوانی های سیستمی، آرایه ای از اشاره گرها به توابع است که در فایل syscall.c قرار دارد (خط ۳۶۷۲). هر کدام از فراخوانی های سیستمی، خود، وظیفه دریافت پارامتر را دارند. ابتدا مختصری راجع به فراخوانی توابع در سطح زبان اسمبلی توضیح داده خواهد شد. فراخوانی توابع در کد اسمبلی شامل دو بخش زیر است:

(گام ۱) ایجاد لیستی از پارامترها بر روی پشته. دقت شود پشته از آدرس بزرگتر به آدرس کوچکتر پر می شود.

ترتیب Push شدن روی پشته:

ابتدا پارامتر آخر، سپس پارامتر یکی مانده به آخر و در نهایت پارامتر نخست. به طور مثال کد اسمبلی کامپایل شده منجر به چنین وضعیتی در پشته سطح کاربر برای تابع $f(a,b,c)$ می شود:

esp+8	C
esp+4	B
esp	A

(گام ۲) فراخوانی دستور اسمبلی معادل call که منجر به Push شدن محتوای کنونی اشاره گر دستورالعمل (eip) بر روی پشته می گردد. محتوای کنونی مربوط به اولین دستورالعمل بعد از تابع فراخوانی شده است. به این ترتیب پس از اتمام اجرای تابع، آدرس دستورالعمل بعدی که باید اجرا شود روی پشته موجود خواهد بود. به طور مثال برای فراخوانی تابع قبلی پس از اجرای دستورالعمل معادل call وضعیت پشته به صورت زیر خواهد بود:

esp+12	c
esp+8	b
esp+4	a
esp	Ret Addr

در داخل تابع f نیز می توان با استفاده از اشاره گر ابتدای پشته به پارامترها دسترسی داشت. به طور مثال برای دسترسی به b می توان از $8+esp$ استفاده نمود. البته اینها تنها تا زمانی معتبر خواهند بود که تابع f تغییری در محتوای پشته ایجاد نکرده باشد.

در فراخوانی سیستمی در $6xv$ نیز به همین ترتیب پیش از فراخوانی سیستمی پارامترها روی پشته سطح کاربر قرار داده شده اند. به عنوان مثال چنانچه در پروژه یک آزمایشگاه دیده شد، برای فراخوانی سیستمی $exec_sys()$ دو پارامتر $\$argv$ و $\$init$ و آدرس برگشتی صفر به ترتیب روی پشته قرار داده شدند (خطوط ۸۴۱۰ تا ۸۴۱۲). سپس شماره فراخوانی سیستمی که در SYS_exec قرار دارد در ثبات eax نوشته شده و $int \$T_SYSCALL$ جهت اجرای تله فراخوانی سیستمی اجرا شد. $exec_sys()$ می تواند مشابه آنچه در مورد تابع $f()$ ذکر شد به پارامترهای فراخوانی سیستمی دسترسی پیدا کند. به این منظور در $6xv$ توابعی مانند $argint()$ و $argptr()$ ارائه شده است. پس از دسترسی فراخوانی سیستمی به پارامترهای مورد نظر، امکان اجرای آن فراهم می گردد.

پرسش 5: با توجه به توضیحاتی که در مورد نحوه قرارگیری پارامترهای فراخوانی سیستمی بر روی پشته و نحوه دسترسی به آنها از طریق توابعی مانند $argptr$ و $argint$ داده شده است، توضیح دهید که این توابع چه نقشی در بازیابی پارامترهای فراخوانی سیستمی دارند. به خصوص، چرا تابع $argptr$ باید بازه های آدرس ورودی را بررسی کند؟ در صورت عدم انجام این بررسی ها، چه نوع مشکلات امنیتی (مانند دسترسی به حافظه خارج از محدوده مجاز) ممکن است ایجاد شود؟ به عنوان مثال، شرح دهید که چگونه نبود این بررسی ها در فراخوانی سیستمی $read_sys$ می تواند باعث بروز اختلال یا آسیب پذیری در سیستم شود.

شیوه فراخوانی فراخوانی های سیستمی جزئی از واسط باینری برنامه های کاربردی (ABI¹⁷) سیستم عامل روی یک معماری پردازنده است. به عنوان مثال در سیستم عامل لینوکس در معماری 6xv پارامترهای فراخوانی سیستمی به ترتیب در ثباتهای ebx, ecx, edx, esi, edi و ebp قرار داده می شوند¹⁸. ضمن این که طبق این ABI، نباید مقادیر ثباتهای ebx, esi, edi و ebp پس از فراخوانی تغییر کنند. لذا باید مقادیر این ثباتها پیش از فراخوانی فراخوانی سیستمی در مکانی ذخیره شده و پس از اتمام آن بازیابی گردند تا ABI محقق شود. این اطلاعات و شیوه فراخوانی های سیستمی را میتوان در فایل های زیر از کد منبع glibc مشاهده نمود¹⁹.

sysdeps/unix/sysv/linux/i386/syscall.S

sysdeps/unix/sysv/linux/i386/sysdep.h

به این ترتیب در لینوکس برخلاف 6xv پارامترهای فراخوانی سیستمی در ثبات منتقل می گردند. یعنی در لینوکس در سطح اسمبلی، ابتدا توابع پوشاننده پارامترها را در پشته منتقل نموده و سپس پیش از فراخوانی فراخوانی سیستمی، این پارامترها ضمن جلوگیری از دست رفتن محتوای ثباتها، در آنها کپی می گردند.

در هنگام تحویل سوالاتی از سازوکار فراخوانی سیستمی پرسیده می شود.

بررسی گام های اجرای فراخوانی سیستمی در سطح کرنل توسط gdb

در این قسمت با توجه به توضیحاتی که تا الان داده شده است، قسمتی از روند اجرای یک سیستم کال را در سطح هسته بررسی خواهیم کرد. ابتدا یک برنامه ساده سطح کاربر بنویسید که بتوان از طریق آن، فراخوانی های سیستمی (getpid() در 6xv را اجرا کرد. یک نقطه توقف (breakpoint) در ابتدای تابع syscall قرار دهید. حال برنامه سطح کاربر نوشته شده را اجرا کنید. زمانی که به نقطه توقف برخورد کرد، دستور bt را در gdb اجرا کنید. توضیح کاربرد این دستور، تصویر خروجی آن و تحلیل کامل تصویر خروجی را در گزارش کار ثبت کنید.

¹⁷ Application Binary Interface

¹⁸ فرض این است که حداکثر ۶ پارامتر ارسال می شود.

¹⁹ مسیرها مربوط به glibc-2.26 است.

حال دستور down (توضیح کارکرد این دستور را نیز در گزارش ذکر کنید) را در gdb اجرا کنید. محتوای رجیستر eax را که در tf می‌باشد، چاپ کنید. آیا مقداری که مشاهده می‌کنید، برابر با شماره فراخوانی سیستمی getpid() می‌باشد؟ علت را در گزارش کار توضیح دهید.

چند بار دستور c را در gdb اجرا کنید تا در نهایت، محتوای رجیستر eax، شماره فراخوانی سیستمی getpid() را در خود داشته باشد.

دقت کنید می‌توانید در ابتدا دستور src layout را اجرا کنید تا کد c در ترمینال gdb نشان داده شود و شاید در تحلیل مراحل، کمکتان کند.

پیاده‌سازی فراخوانی‌های سیستمی

در این آزمایش با پیاده‌سازی فراخوانی‌های سیستمی، اضافه کردن آن‌ها به هسته 6xv را فرا می‌گیرید. در این فراخوانی‌ها که در ادامه توضیح داده می‌شود، پردازش‌هایی بر پردازش‌های موجود در هسته و فراخوانی‌های سیستمی صدا زده شده توسط آنها انجام می‌شود که از سطح کاربر قابل انجام نیست. شما باید اطلاعات فراخوانی‌های سیستمی مختلفی که توسط پردازش‌ها صدا زده می‌شوند را ذخیره کنید و روی آنها عملیاتی انجام دهید. تمامی مراحل کار باید در گزارش کار همراه با فایل‌هایی که آپلود می‌کنید موجود باشند.

نحوه اضافه کردن فراخوانی‌های سیستمی

برای انجام این کار لینک و مستندات زیادی در اینترنت و منابع دیگر موجود است. شما باید چند فایل را برای اضافه کردن فراخوانی‌های سیستمی در 6xv تغییر دهید. برای این که با این فایل‌ها بیشتر آشنا شوید، پیاده‌سازی فراخوانی‌های سیستمی موجود را در 6xv مطالعه کنید. این فایل‌ها شامل user.h، syscall.c، syscall.h و ... است. گزارشی که ارائه می‌دهید باید شامل تمامی مراحل اضافه کردن فراخوانی‌های سیستمی و همچنین مستندات خواسته‌شده در مراحل بعد باشد.

نحوه ذخیره اطلاعات پردازشها در هسته

پردازشها در سیستم عامل 6xv پس از درخواست یک پردازش دیگر توسط هسته ساخته می‌شوند. در این صورت هسته نیاز دارد تا اولین پردازش را خودش اجرا کند. هسته 6xv برای نگهداری هر پردازش یک ساختار داده ساده دارد که در یک لیست مدیریت می‌شود. هر پردازش اطلاعاتی از قبیل شناسه واحد خود²⁰ که توسط آن شناخته می‌شود، پردازش والد و غیره را در ساختار خود دارد. برای ذخیره کردن اطلاعات بیشتر، می‌توان داده‌ها را به این ساختار داده اضافه کرد.

حال قصد داریم تا با استفاده از چند فراخوانی سیستمی روند صدا زده شدن فراخوانی‌های سیستمی توسط پردازشها را بررسی کنیم و اطلاعات استفاده شده و دستکاری شده توسط آنها را نمایش دهیم. هدف از این بخش آشنایی با بخش‌های مختلف عملکرد فراخوانی‌های سیستمی است:

1. بخش اول پیاده سازی فراخوانی سیستمی مولد اعداد تصادفی:

می‌خواهیم به xv6 یک مولد اعداد تصادفی اضافه کنیم. برای این کار به دو فراخوانی سیستمی نیاز داریم :

(1) فراخوانی سیستمی setSeed

این فراخوانی seed را برای مولد اعداد تصادفی تنظیم می‌کند (Seed عدد اولیه‌ای است که به الگوریتم تولید اعداد شبه تصادفی داده می‌شود تا دنباله اعداد «به ظاهر رندوم» ولی قابل تکرار تولید کند).

```
int setSeed(void)
```

این فراخوانی باید با استفاده از وضعیت فعلی سیستم، مقدار اولیه‌ی مولد اعداد شبه تصادفی را در هسته تنظیم کند. فرمول تولید seed به صورت زیر است:

```
seed = (ticks << 20) ^ (pid << 10) ^ (stack_addr & 0x3FF)
```

²⁰PID

Ticks :	زمان سیستم
PID :	شناسه فرآیند جاری
Stack_addr:	آدرس متغیر محلی در هسته

در صورت موفقیت مقدار 0 و در صورت خطا مقدار -1 برگرداند.

(2) فراخوانی سیستمی getRandomNumber

```
int getRandomNumber(int n, int *buf)
```

این فراخوانی باید با استفاده از seed فعلی، n عدد شبه تصادفی تولید کرده و در بافر buf قرار دهد. فرض کنید $n < 15$ است.

برای تولید هر عدد جدید، state مولد باید به شکل زیر به روزرسانی شود:

```
random_state = (random_state * 1103515245 + 12345) & 0x7fffffff
```

seed نقطه‌ی شروع مولد اعداد شبه تصادفی است. اولین عدد تولید شده از روی seed محاسبه می‌شود و هر عدد بعدی از روی مقدار قبلی state ساخته می‌شود. بنابراین اگر مقدار اولیه‌ی seed یکسان باشد، دنباله‌ی اعداد تولید شده نیز یکسان خواهد بود. اگر seed تغییر کند، معمولاً دنباله‌ی خروجی نیز تغییر می‌کند.

در صورت موفقیت مقدار 0 و در صورت خطا مقدار -1 برگرداند.

توجه : در صورت استفاده از متغیرهای سراسری مشترک در هسته، لازم است بخش‌های مربوط به خواندن و تغییر آن‌ها با acquire() و release() محافظت شوند تا از تداخل دسترسی‌ها و ناسازگاری داده جلوگیری شود (در این پروژه، seed یا random_state نمونه‌ای از این متغیرهاست).

برای آزمایش این بخش، برنامه‌ای در سطح کاربر بنویسید که با استفاده از فراخوانی‌های سیستمی setSeed و getRandomNumber، یک تاس مجازی را شبیه‌سازی کند. این برنامه باید تعداد دفعات تاس انداختن را از ورودی دریافت کند.

پرسش‌ها:

1. در مورد مولد اعداد تصادفی در windows و linux مطالعه کنید. چرا مولد اعداد تصادفی را در هسته سیستم عامل پیاده سازی می‌شود؟
2. فرض کنید دو بار پشت سر هم setSeed را اجرا می‌کنیم. انتظار داریم خروجی تغییری کند؟ چرا؟

2. بخش دوم : پیاده سازی فراخوانی سیستمی نمایش برخی اطلاعات پردازش

در این بخش، می‌خواهیم فراخوانی سیستمی ای پیاده سازی کنیم که اطلاعات پردازش ای که شناسه آن را در ورودی گرفته، نمایش دهد.

```
int process_information(int pid)
```

این فراخوانی سیستمی باید اطلاعات زیر را برای پردازشی موردنظر نمایش دهد:

- شناسه‌ی پردازش
- شناسه‌ی والد پردازش
- شناسه‌ی فرزندان پردازش
- شناسه‌ی پردازش‌هایی که با آن والد مشترک دارند
- عمق پردازش در درخت پردازش‌ها
- وضعیت فعلی پردازش

خروجی باید با قالب زیر نمایش داده شود:

```
Process_id:
Parent_id:
Children's_id: child1, child2, ..., child n
Siblings_id: sibling1, sibling2, ..., sibling n
Depth:
State:
```

برای تست این فراخوانی سیستمی یک برنامه سطح کاربر بنویسید که در آن هر پردازش چند فرزند و چند برادر داشته باشد.

نکات

- اگر pid موردنظر یافت نشود، این فراخوانی باید مقدار 1-برگرداند .
- در صورت موفقیت، مقدار 0 برگرداند .
- این فراخوانی باید در هسته‌ی سیستم‌عامل پیاده‌سازی شود و اطلاعات را مستقیماً از جدول پردازش‌ها استخراج کند.

پرسش‌ها:

1. در مورد state پردازش‌ها در xv6 مطالعه کنید. در مورد هر state توضیح دهید.
2. اگر پردازش‌ای شناسه خودش را بدهد، در قسمت state چه نتیجه‌ای نمایش داده می‌شود؟
3. آیا می‌توانستیم process information را در فضای کاربر پیاده‌سازی کنیم؟ مزایا و معایب آن را باهم مقایسه کنید.

3. بخش سوم : پیاده سازی sort در هسته سیستم عامل

در این بخش، می‌خواهیم یک فراخوانی سیستمی برای مرتب‌سازی اعداد پیاده‌سازی کنیم.

```
int sort_numbers(const char *src_file)
```

این فراخوانی سیستمی باید نام یک فایل را به عنوان ورودی بگیرد، محتوای آن را بخواند، اعداد موجود در فایل را مرتب کند، و نتیجه را در یک فایل جدید در همان مسیر ذخیره کند.

این فراخوانی سیستمی باید در هسته سیستم عامل پیاده سازی شود.

برای آزمایش این فراخوانی سیستمی، یک برنامه‌ی سطح کاربر بنویسید که نام فایل ورودی را از کاربر گرفته و فراخوانی سیستمی sort را اجرا کند و مدت زمان مرتب‌سازی را اندازه بگیرید.

بخش 3.2

همان الگوریتم مرتب‌سازی را این بار در یک برنامه‌ی سطح کاربر پیاده‌سازی کنید و روی همان ورودی اجرا کنید و مدت زمان مرتب‌سازی را اندازه بگیرید.

بخش 3.3

مدت زمان اجرای دو برنامه سطح کاربر بالا را باهم مقایسه کنید.

نکات:

- اگر فایل ورودی وجود نداشته باشد، فراخوانی sort باید مقدار 1 برگرداند.
- در صورت موفقیت، مقدار 0 برگرداند.
- ورودی شامل چند خط است. در هر خط دقیقاً یک عدد صحیح قرار دارد.
- از الگوریتم‌های مرتب‌سازی بازگشتی استفاده نکنید.

پرسش‌ها:

1. تعداد تغییر mode ها بین فضای کاربر و هسته را در هر بخش محاسبه کنید و در گزارش بنویسید.
2. به نظر شما آیا نیاز بود که sort رو در فضای هسته پیاده سازی کنیم؟ پاسخ خود را توضیح دهید.
3. در مورد دستور sort در Linux مطالعه کنید. این دستور در فضای کاربر پیاده سازی شده یا فضای هسته؟
4. کرنل مسئول چه کارهایی است؟

5. چرا نباید از الگوریتم های مرتب سازی بازگشتی استفاده کنیم؟ در صورت استفاده چه مشکلی رخ خواهد داد؟

سایر نکات

- پاسخ به سوالات تئوری را در گزارش خود بیاورید و در کوتاه ترین حالت ممکن پاسخ دهید همچنین برای بخش های عملی صرفاً نتایج را قرار دهید.
- همه افراد باید به پروژه آپلود شده توسط گروه مسلط باشند و لزوماً نمره افراد یک گروه با یکدیگر برابر نیست.
- در صورت مشاهده هرگونه مشابهت بین کدها یا گزارش دو گروه، به هر دو گروه نمره 0 و حتی نمره منفی تعلق خواهد گرفت.

موفق باشید